ARCHITECTURES OF CONVOLUTIONAL NEURAL NETWORK

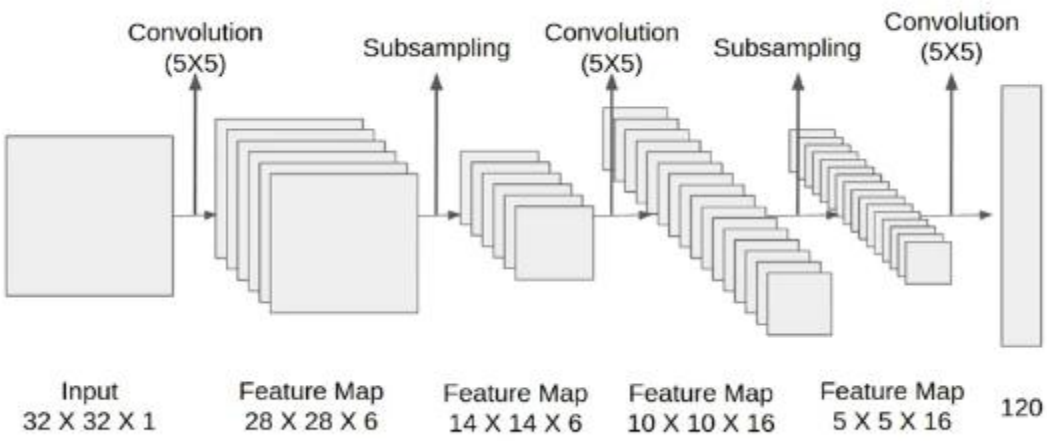
Convolutional Neural Networks (CNN) are a specialized class of deep neural networks, primarily designed for processing visual data such as images and videos. CNN architectures are inspired by the human visual system. Using Different layers such as convolutional layers, pooling layers and fully connected layers, CNN captures spatial hierarchies of the images.

Over the years several architectures are developed to improve accuracy, efficiency and depth, addressing challenges such as over fitting, vanishing/exploding gradients, and computational complexity. Some of the popular architectures of CNN are,

1. LeNet
2. AlexNet
3. VGG
4. ResNet
5. GoogLeNet
6. DenseNet

LeNet

LeNet CNN was proposed by Yann LeCun in the year 1998. Being one of the earliest and most influencial CNN architecture, it was primalily designed for handwritten, digital recognition such as numbers on bank checks and postal codes.



1. Input Layer:
    - Input image size: 32 X 32 grayscale image.
    - Image is normalized before processing.
2. Convolutional Layer (C1):
    - Applies multiple filters to extract basic features.
    - Feature maps detect edges and simple patterns.
    - Output: 28 X 28 with depth of 6.
3. Subsampling / Pooling Layer (S1):

    - Reduces special dimensions.
    - Helps in reducing the computation and overfitting.
    - Output: 14 X 14 with depth of 6
4. Convolution Layer (C3):
    - Extracts more complex features.
    - Builds on patterns learned in earlier layer.
    - Output: 5 X 5 with depth of 16
5. Pooling Layer (S4):
    - Further down-samples feature maps.
    - Improves robustness to small distortions.
    - Output: 5 X 5 with depth of 16
6. Fully connected Layers:
    - Combines extracted features.
    - Performs final classifications.
7. Output Layer
    - Uses softmax activation function.
    - Outputs contains 10 neurons for digits from 0 to 9.

LeNet uses ReLU activation function for pooling/ subsampling. Earlier tanh (hyperbolic tangent) was used at the time of LeNet's development. LeNet has a total 60,000 trainable parameters (weights and biases) across convolutional and fully connected layers.

LeNet introduced convolutional layer and used shared weights which reduced the number of parameters compared to fully connected networks.

Advantages of LeNet:

- Simple and easy to understand
- Efficient for small image datasets
- Laid the foundation for modern CMMs
- Performs well for digit recognition tasks

Limitations of LeNet:

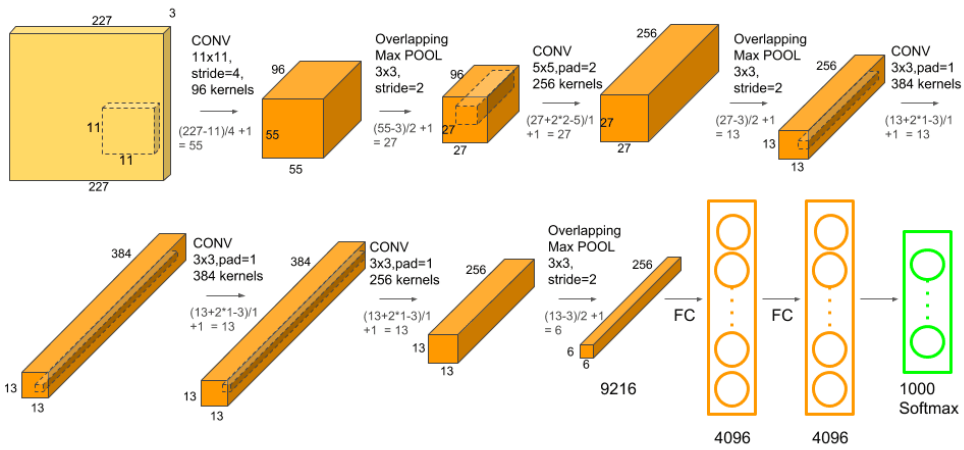
- Not suitable for large or complex datasets
- Uses tanh instead of faster ReLU activation
- Limited depth compared to modern CNNs
- Not designed for color images

Applications of LeNet:

- Handwritten digit recognition
- Optical Character Recognition
- Basic image classification tasks

AlexNet

AlexNet architecture was introduced by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hilton in the year 2012. It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2012) with a significant margin, demonstrating the effectiveness of deep learning for large-scale image classification tasks.



1. Convolution Layer 1:
    - Fetches input of 227 X 227 RGB image
    - With 96 filters of size 11 X 11, strides: 4, input is convoluted
    - This captures low-level features such as edges and textures.
    - Output: 55 X 55 with depth 96
2. Max Pooling Layer:
    - Pool size 3 X 3, strides: 2
    - Output: 27 X 27 of depth 96
3. Convolution Layer 2:
    - 256 Filters of size 5 X 5
    - Padding is used to preserve spatial size
    - Output: 27 X 27 with depth 256
    - Extracts more complex features
4. Max Pooling Layer:
    - Uses same configurations as previous pooling layer
    - Output: 13 X 13 with depth 256
5. Convolution Layers 3, 4, 5:
    - 384, 384 and 256 filters respectively
    - Filter size: 3 X 3
    - Output: 13 X 13 with depth 384, 384 and 256 respectively
    - Captures high level features
6. Fully Connected Layers:
    - The first two layers has 4096 neurons each

- Final FC layer has 1000 neurons
- It performs classification

AlexNet uses the Rectified Linear Unit (ReLU) activation function, which significantly accelerates training compared to sigmoid and tanh functions. It uses three regularization techniques: Dropout, Data Augmentation, and Local Response Normalization (LRN). **Dropout** is used in fully connected layers to prevent over-fitting. **Data Augmentation** is used to increase training data. **LRN** is used to improve generalization.

Advantages of AlexNet:

- High accuracy on large datasets
- Faster convergence due to ReLU activation function
- Reduced over-fitting.

Limitations of AlexNet:

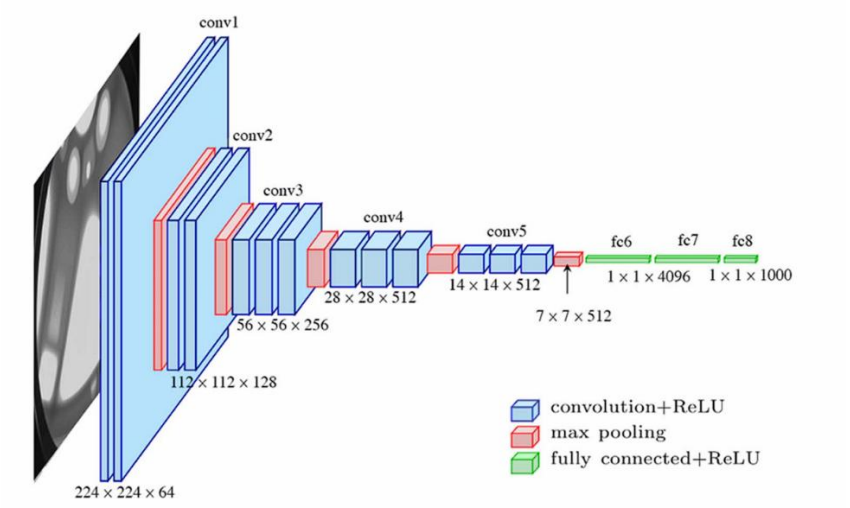
- Large number of parameters approximately 60 millions
- High memory and computational cost
- Not suitable for small datasets

Applications of AlexNet:

- Image classification
- Object detection
- Feature extraction
- Transfer learning

Visual Geometry Group (VGG) Network

The VGG network was developed by the Visual Geometry Group at the University of Oxford in the year 2014. VGG gained recognition for its simplicity and depth, proving that increasing network depth using small convolution filters can significantly improve image classification performance.



VGG uses multiple 3 X 3 convolutions instead of large filters such as 5X5 / 7X7filters. This allows the network to learn more complex features with fewer parameters and more non-linearity.

1. Input Layer:
    - Input image size: 224 X 224 X 3 (RGB image)
    - Images are pre-processed by mean subtraction and normalization
    - This fixed input size allows consistent feature extraction across layers
2. Block 1:
    - 2 convolution layers: 64 filters with kernel size 3X3 and stride: 1
    - Output: 224 X 224 with depth 64
3. Block 2:
    - 1 max pooling layer with pool size: 2X2 and strides: 2 to reduce spatial dimensions by half
    - 2 convolution layers: 128 filters with kernel size: 3X3 and stride: 1
    - Output: 112 X 112 with depth 128
4. Block 3:
    - 1 max pooling layer with pool size: 2X2 and strides: 2 to reduce spatial dimensions by half
    - 3 convolution layers: 256 filters with kernel size: 3X3 and stride: 1
    - Output: 56 X 56 with depth 256
5. Block 4:
    - 1 max pooling layer with pool size: 2X2 and strides: 2 to reduce spatial dimensions by half
    - 3 convolution layers: 512 filters with kernel size: 3X3 and stride: 1
    - Output: 28 X 28 with depth 512
6. Block 5:
    - 1 max pooling layer with pool size: 2X2 and strides: 2 to reduce spatial dimensions by half
    - 3 convolution layers: 512 filters with kernel size: 3X3 and stride: 1
    - Followed by 1 max pooling layer with pool size: 2X2 and strides: 2 to reduce spatial dimensions by half
    - Output: 7 X 7 with depth 512
7. Fully Connected Layers:
    - FC1 and FC2 with 4096 neurons each
    - FC3 with 1000 neurons for classification
    - Final layer uses Softmax activation function for multiclass classification.

VGG uses Dropout with typical rate of 0.5 in fully connected layers. VGG has 2 variants with **VGG-16**, containing 13 convolutional layers and 3 fully connected layers. VGG-19, containing 16 convolutional layers and 3 fully connected layers. Both follows same design philosophy, differing only in depth.

Advantages of VGG:

- Easy to understand and implement
- Excellent performance on large image datasets
- Widely used for transfer learning

Limitations of VGG:

- Large number of parameters, approximately 138 million
- High memory consumption
- Slow training inference
- Not suitable for resource-constrained environments

Applications of VGG:

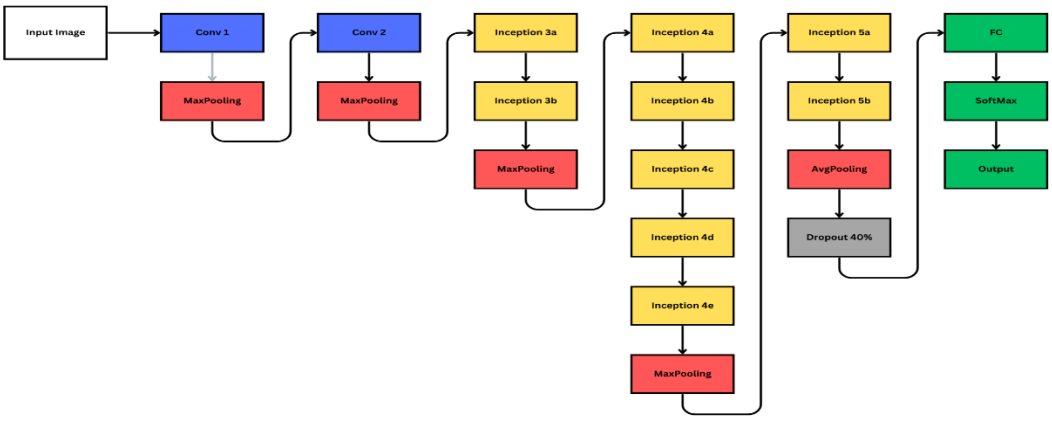
- Image classification
- Object recognition
- Feature extraction
- Transfer learning in computer vision tasks

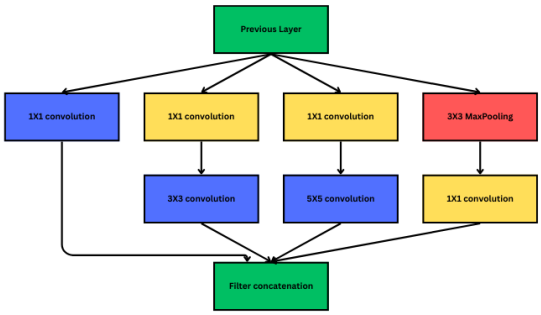
GoogLeNet (Inception) Network

GoogLeNet, also known as Inception v1, was introduced by Google in the year 2014 and won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2014). The main goal of the GoogLeNet was to build a deep yet computational efficient convolutional neural network, using novel structure called Inception module.

GoogLeNet is based on the idea that, important visual information exists at multiple scale. Instead of choosing a single filter size, GoogLeNet appies multiple convolution filters in parallel and lets the network decide the most relevant features, such as

- Multi-scale feature extraction
- Efficient use of computation
- Reduction of parameters
- Increased network depth without overfitting





1. Input Layer:
    - Input image size: 224 X 224 X 3 (RGB image)
    - Input images are normalized before processing
2. Inception Module:
    - Inception is the heart of GoogLeNet. It performs multiple operations in parallel on the same input.
    - 1X1, 3X3, 5X5 convolutions with 3X3 max pooling operations are concatenated depth wise to form the final output of the module.
3. Role of 1X1 Convolutions:
    - Reduce number of input channel
    - Decrease computational cost
    - Reduce number of parameters
    - Add non-linearity with ReLU.
4. Overall Architecture:
    - Approximately 22 layers deep excluding pooling layers
    - Composed of stacked Inception modules
    - No large fully connected layers
    - Use global average pooling instead of traditional FC layers
5. Pooling and Downsampling:
    - Max pooling layers are placed between groups of inception modules
    - Reduce spatial resolution
    - Control overfitting and computation
6. Auxiliary Classifiers:
    - Auxiliary classifiers are introduced at the intermediate layers during training
    - This layer improves gradient flow
    - Acts as regularizers
    - Reduce vanishing gradient problem
    - Activated only during training and removed during inference
7. Final Classification Layer:
    - Uses global average pooling
    - Followed by Softmax layer
    - Produces probability distribution across 1000 classes

Advantages of GoogLeNet:

- High accuracy with fewer parameters
- Reduced overfitting
- Efficient memory usage
- Faster inference compared to VGG

Limitations of GoogLeNet:

- Complex architecture
- Difficult to modify
- Less intuitive than simple CNNs

Applications of GoogLeNet:

- Image classifications
- Object detection
- Face recognition
- Medical image analysis

ResNet Network

Residual Network (ResNet) was introduced by Microsoft in the year 2015 by Kaiming He and his team. It was designed to solve major problem in deep neural network, that is, vanishing gradient, which occurs when the network becomes very deep. ResNet made it possible to train extremely deep networks such as ResNet-50, and ResNet-152, achieving outstanding performance in image recognition task.

The core idea of ResNet is, instead of learning a direct mapping from the input to output, it learns a residual function

$$H(x) = F(x) + x$$

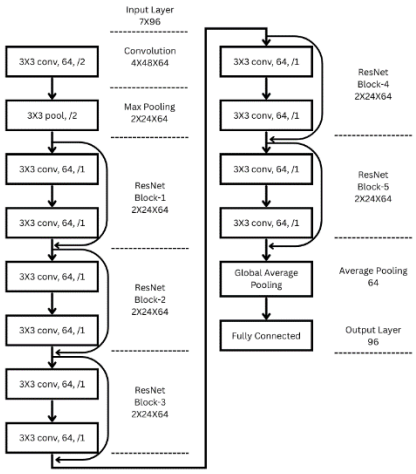
Where:

- x is input
- f(x) is the residual mapping learned by the network
- H(x) is the output

This is implemented using skip connections, where the input bypasses one or more layers and directly added to the output.

ResNet Has two types of residual blocks

1. Identity Block:
    - Input and output dimensions are same
    - Shortcut directly adds input to output
2. Convolutional Block
    - Used when dimensions change
    - Shortcut includes a convolution layer to match dimensions



1. Architecture:
   ResNet follows a structured design:
   - Initial convolution and max pooling
   - Series of residual blocks
   - Global average pooling
   - Fully connected layer
   - Softmax output layer
2. Depth variants:
   - ResNet-18 and ResNet-34 uses basic residual network
   - ResNet-50, ResNet-101, ResNet- uses bottleneck residual blocks
   - Bottleneck Block uses:
     1X1 convolution for dimensionality reduction
     3X3 convolution
     1X1 convolution for dimension reduction
   - This reduces computational cost while maintaining depth
3. Activation Function:
   - Uses ReLU activation function
   - It enables faster training and reduces vanishing gradient
4. Batch Normalization:
   - Applied after convolution layers
   - Stabilizes learning
   - Allows higher learning rate
   - Improves convergence
5. Pooling and Down-sampling:
   - Max pooling is used in the early stage of the network
   - Pool size of 3X3 with stride 2 is used
   - This gradually reduces spatial dimensions

6. Final Classification Layer:
    - The final layer uses global average pooling
    - Followed by fully connected later
    - Softmax activation is used for classification

Advantages of ResNet:

- Solves vanishing gradient problem
- Enables very deep network
- High accuracy
- Faster convergence

Limitations of ResNet:

- Increased complexity
- Higher computational cost for deeper versions
- Not always optimal for small datasets

Applications of ResNet:

- Image classifications
- Object detection
- Face Recognition
- Medical imaging
- Autonomous driving

DenseNet Network

Densely Connected Convolutional Network (DenseNet) was introduced by Gao Huang and team, in the year 2017. DenseNet was designed to further improve information flow, and feature reuse in deep neural networks. Unlike traditional CNNs and ResNet, DenseNet connects each layer to every other layers in feed-forward manner, making it highly efficient and powerful.

As the network becomes deeper, they face vanishing gradient problem, redundant feature learning, inefficient parameter usage. DenseNet address these challenges by densely connecting the layers, ensuring maximum information and gradient flow throughout the network.

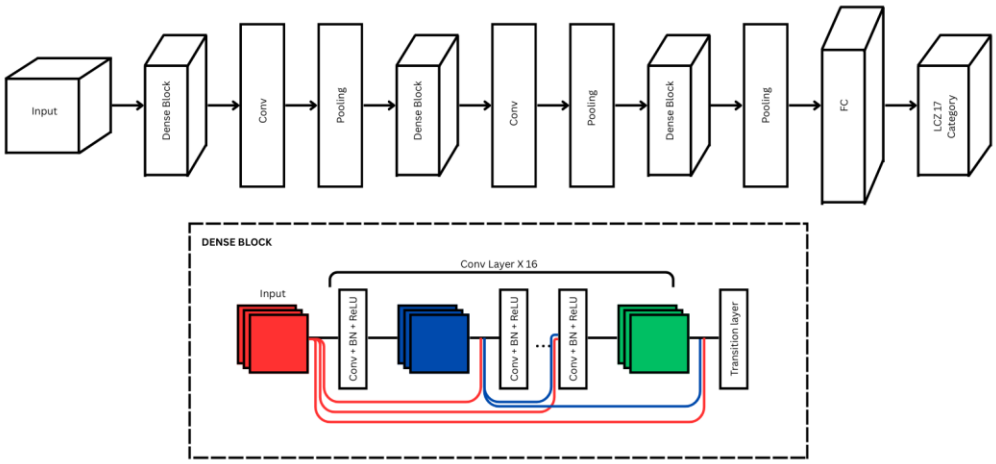
Core Idea of Dense Connectivity:

In DenseNet, each layer receives the feature maps of all preceding layers as input, and its own output is passed to all subsequent layers.

$$x1 = H1([x_{0,}x_1, x_2,..., x_{l-1}])$$

where,

- $[x_{0,}x_1, x_2,..., x_{l-1}]$ represents concatenation
- H1 is a composite function of batch normalization, ReLU and convolution



Dense Block:

Dense blocks are the building blocks of DenseNet architecture. The structure of Dense block is:

- Multiple convolution layers
- Each layer takes all previous feature maps as input
- Feature maps are concatenated

This implementation encourages feature reuse, reduce the need to relearn redundant features and improve learning efficiency.

Growth Rate:

The growth rate (k) defines how many new feature maps each layer adds. If growth rate = K, then, each layer outputs ‘k’ number of feature maps. Total feature maps increase linearity with depth. Smaller growth rate results in more compact models.

Bottleneck Layers:

To reduce computational cost, DenseNet uses bottleneck layers. It uses 1X1 convolution before 3X3 convolution. This reduces the number of input feature maps and improves efficiency.

Transition Layers:

The transition layers are placed before dense blocks to reduce spatial dimension, compress feature maps and control complexity. Transition layer components include:

- Batch Normalization
- 1X1 convolution
- 2X2 average pooling

DenseNet Structure:

1. Initial convolution Layer
2. Dense Block followed by Transition layer (X2)

3. Dense block
4. Global Average pooling
5. Fully connected layer
6. Softmax output

DenseNet architecture uses ReLU activation function to promote faster convergence and batch normalization is applied befire convolutions to stabilize training and improve gradient flow

Advantages of DenseNet:

- Reduced vanishing gradient problem
- High accuracy with fewer parameters
- Efficient learning
- Strong performance on small datasets

Limitations of DenseNet:

- High memory usage
- Complex architecture
- Slower inference in very deep network

Applications of DenseNet:

- Image classification
- Medical imaging
- Object recognition
- Research and academic use